

LABORATORIO 14: Simulación VR Monoscópica con Posición Fija

1. Objetivo de aprendizaje

Lab 14 es una **extensión arquitectónica** del Lab 13. El alumno modificará la base de código para construir una experiencia de **Realidad Virtual monoscópica** (pantalla completa, mundo cerrado) donde:

- El usuario **no puede caminar** en el espacio virtual (posición bloqueada).
- El usuario **sí puede girar** el celular para mirar alrededor (rotación libre).
- El fondo es un **skybox equirectangular opaco** que tapa completamente la cámara real.
- Se mantiene el tracking de piso (`local-floor`) para que el modelo quede anclado a $Y=0$.

Este laboratorio enseña el concepto de **diseño bajo restricción**: qué pasa cuando tomamos un sistema diseñado para movimiento libre (6DoF) y le imponemos artificialmente una limitación (3DoF de posición).

2. ¿Por qué es una extensión y no un proyecto nuevo?

Lab 14 reutiliza **exactamente la misma arquitectura** del Lab 13, con cuatro modificaciones puntuales:

Aspecto	Lab 13 (AR)	Lab 14 (VR monoscópico)
Fondo	Cámara real (passthrough)	Skybox equirectangular opaco
Posición del usuario	Libre (caminar)	Fija en $(0, 1.6, 0)$
Renderer	<code>alpha: true, autoClear: false</code>	<code>alpha: false, autoClear: true</code>
View Matrix	Directa de WebXR	Reconstruida manualmente con posición forzada

Todo lo demás —sesión WebXR, espacio de referencia, carga de modelo, luces, bounds— permanece idéntico.

3. Fundamento teórico: De 6DoF a "3DoF de posición"

En el Lab 13, ARCore entrega la posición real del usuario en la habitación. Si camina 2 metros al norte, la cámara se traslada 2 metros al norte. Eso es 6DoF completo.

En el Lab 14, queremos que el usuario **gire** el celular para explorar el entorno, pero **no se mueva** cuando camine físicamente. Esto simula un visor de realidad virtual estacionario (como un binocular digital o un planetario portátil).

La solución no es "desactivar el tracking de posición" (ARCore siempre lo entrega). La solución es **ignorar la traslación** que entrega ARCore y reconstruir la View Matrix usando solo la rotación del dispositivo, combinada con una posición fija artificial.

4. Explicación del código: diferencias respecto al Lab 13

4.1 Renderer y fondo

// LAB 13:

```
renderer = new THREE.WebGLRenderer({ alpha: true });  
renderer.autoClear = false;  
scene.background = null;
```

// LAB 14:

```
renderer = new THREE.WebGLRenderer({ alpha: false });  
renderer.autoClear = true; // por defecto  
scene.background = texture; // skybox opaco
```

En el Lab 14 volvemos al comportamiento estándar de Three.js: limpiar el framebuffer completo y pintar un fondo opaco. La cámara real ya no se ve.

4.2 Skybox equirectangular

```
const texLoader = new THREE.TextureLoader();  
texLoader.load('textures/textura.png', (texture) => {  
    texture.mapping = THREE.EquirectangularReflectionMapping;  
    texture.colorSpace = THREE.SRGBColorSpace;
```

```

scene.background = texture;

scene.environment = texture;


const geo = new THREE.SphereGeometry(500, 60, 40);
geo.scale(-1, 1, 1);
const mat = new THREE.MeshBasicMaterial({
  map: texture,
  side: THREE.BackSide,
  depthWrite: false
});
const skyMesh = new THREE.Mesh(geo, mat);
skyMesh.renderOrder = -1000;
scene.add(skyMesh);
});

```

El skybox es una esfera gigante (`radio = 500`) con la textura equirectangular mapeada en su interior (`scale(-1, 1, 1)` invierte las normales para que se vea desde adentro). `depthWrite: false` evita que el skybox interfiera con el z-buffer de los objetos cercanos.

4.3 onXRFrame() — La reconstrucción matricial

Esta es la diferencia más importante. En el Lab 13 usábamos la View Matrix directamente:
 // LAB 13:

```

camera.matrixWorldInverse.fromArray(view.transform.inverse.matrix);

```

En el Lab 14 debemos **reconstruirla** para forzar la posición:

// LAB 14:

```
const vm = new THREE.Matrix4().fromArray(view.transform.inverse.matrix);
```

```
// Extraer la rotacion pura (3x3 superior de la view matrix)
```

```
const r00 = vm.elements[0], r10 = vm.elements[1], r20 = vm.elements[2];
```

```
const r01 = vm.elements[4], r11 = vm.elements[5], r21 = vm.elements[6];
```

```
const r02 = vm.elements[8], r12 = vm.elements[9], r22 = vm.elements[10];
```

```
// Posicion fija: (0, 1.6, 0) — altura de ojos de un adulto promedio
```

```
const px = 0, py = 1.6, pz = 0;
```

```
// Calcular la traslacion de la view matrix para esa posicion fija
```

```
//  $t = -R^T * p$ 
```

```
const tx = -(r00 * px + r01 * py + r02 * pz);
```

```
const ty = -(r10 * px + r11 * py + r12 * pz);
```

```
const tz = -(r20 * px + r21 * py + r22 * pz);
```

```
// Construir la view matrix corregida
```

```
const fixedVM = new THREE.Matrix4();
```

```
fixedVM.set(
```

```
    r00, r01, r02, tx,
```

```
    r10, r11, r12, ty,
```

```
    r20, r21, r22, tz,
```

```
    0, 0, 0, 1
```

```
);
```

```
// Aplicar a la camara
```

```
camera.matrixWorldInverse.copy(fixedVM);
```

```
camera.matrixWorld.copy(fixedVM).invert();
```

```
camera.matrix.copy(camera.matrixWorld);
```

¿Qué hace esto matemáticamente?

La View Matrix tiene la forma $[R \mid t]$, donde:

- R es la matriz de rotación 3×3 (orientación del celular).
- t es el vector de traslación.

Normalmente, $t = -R^T * p_{real}$, donde p_{real} es la posición física del usuario entregada por ARCore.

Nosotros **forzamos** $p = (0, 1.6, 0)$ en lugar de p_{real} . Recalculamos t con esa posición artificial. El resultado: la cámara rota exactamente igual que el celular (misma R), pero **nunca se traslada** del punto $(0, 1.6, 0)$.

4.4 ¿Por qué no usamos Euler ni cuaterniones?

Durante el desarrollo de este laboratorio se probaron múltiples aproximaciones:

Enfoque rechazado

Descomponer a Euler
(`setFromQuaternion`)

```
euler.x = -euler.x
```

Mezclar yaw de una matriz y pitch de otra

```
camera.quaternion.copy(quat)  
directo
```

¿Por qué falló?

Gimbal lock y ambigüedad de cuadrante. El pitch se invertía o saltaba 180° .

Funcionaba en algunos rangos de ángulo, pero fallaba al cruzar $\pm 90^\circ$.

Los ejes de Euler no son independientes; mezclar fuentes rompe la ortonormalidad.

Invertía el yaw o el pitch dependiendo de si usábamos `matrix` o `inverse.matrix`.

La solución robusta fue **no descomponer nunca**. Extraer la rotación 3×3 pura y reconstruir la matriz 4×4 completa mediante álgebra lineal garantiza que la orientación se preserve exactamente como la entrega ARCore, sin pérdida de información ni artefactos de cuadrante.

5. Diagrama comparativo: Lab 13 vs Lab 14

LAB 13: AR Passthrough
Fondo: Cámara real del celular Movimiento: Libre (caminar) Sensación: "Hay una maqueta en mi sala" Uso didáctico: AR espacial, anclaje

↓ Misma base, misma sesión, misma arquitectura

LAB 14: VR Monoscópica Fija
Fondo: Skybox opaco (textura.png) Movimiento: Solo girar, no caminar Sensación: "Estoy dentro de un mundo" Uso didáctico: Control de cámara, diseño bajo restricción, matrices

Código QR

